

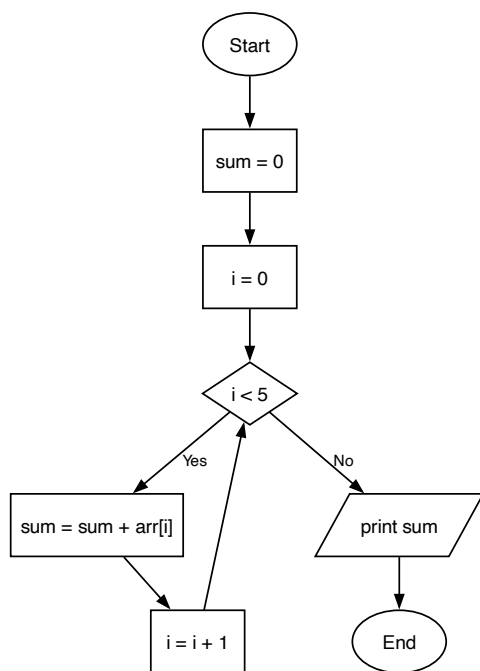
Answer key

Problem 1.

This program adds up all the numbers in a list. It keeps a running total in sum and a position marker i , and walks through the list one item at a time — from position 0 to the last — adding each item to the total as it goes.

Trace this algorithm, taking the list arr to be $[4, 7, 2, 9, 5]$, and complete the trace table.

Reminder: the items of a list have positions counted from 0, so the list $[4, 7, 2, 9, 5]$ has 5 items in positions 0 to 4 — position 0 holds 4, position 1 holds 7, and so on. $arr[i]$ means the item at position i , so $arr[0]$ is the first item.



sum	i	output
0		
0	0	
4	0	
4	1	
11	1	
11	2	
13	2	
13	3	
22	3	
22	4	
27	4	
27	5	
27	5	27

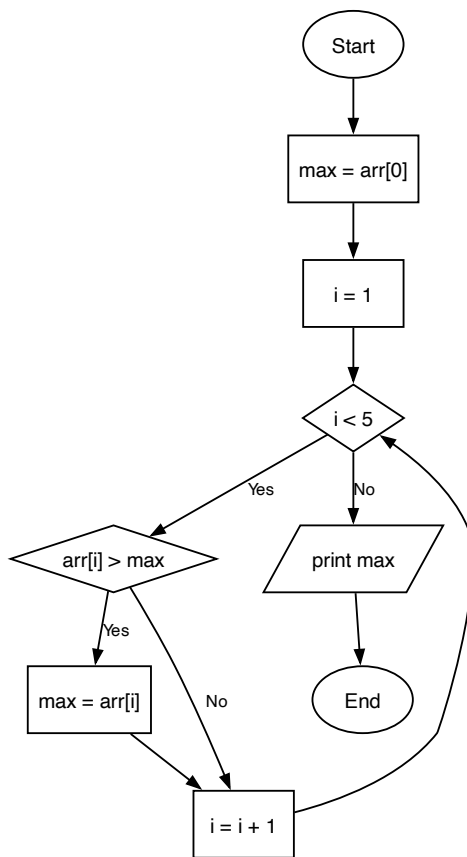
Output: 27

Problem 2.

This program finds the largest number in a list. It begins by assuming the first item (position 0) is the biggest, then walks through the rest of the list: whenever it meets an item bigger than the best seen so far, it remembers that one instead. max holds the largest value found so far.

Trace this algorithm, taking the list arr to be $[3, 9, 2, 12, 7]$, and complete the trace table.

Reminder: the items of a list have positions counted from 0, so the list $[4, 7, 2, 9, 5]$ has 5 items in positions 0 to 4 — position 0 holds 4, position 1 holds 7, and so on. $\text{arr}[i]$ means the item at position i , so $\text{arr}[0]$ is the first item.



max	i	output
3		
3	1	
9	1	
9	2	
9	3	
12	3	
12	4	
12	5	
12	5	12

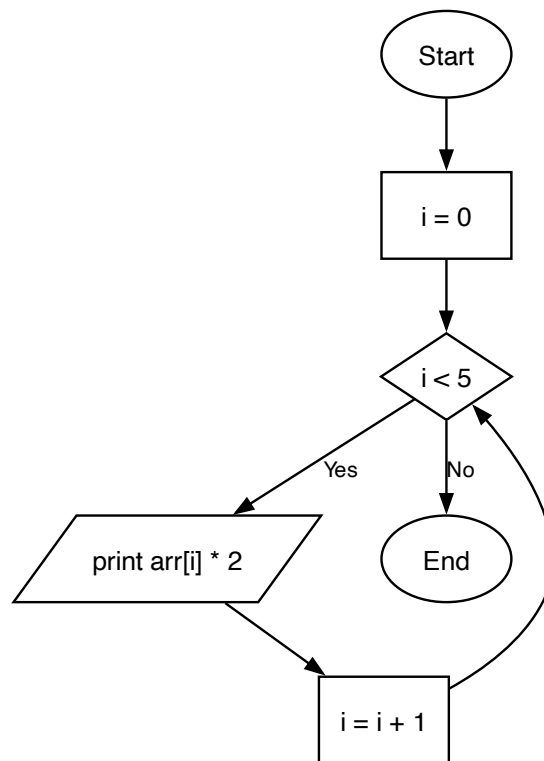
Output: 12

Problem 3.

(i) Expected output for each input:

input <i>arr</i>	output
(3, 8, 1, 6, 4)	6, 16, 2, 12, 8
(10, 0, 5, 2, 7)	20, 0, 10, 4, 14

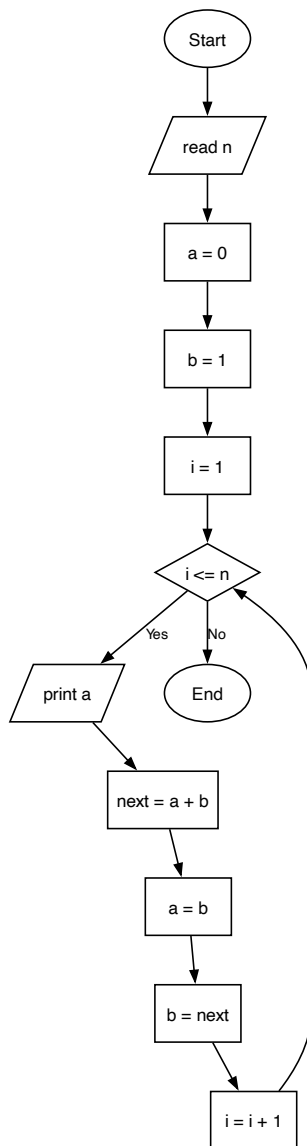
(ii) Flowchart:



Problem 4.

The Fibonacci sequence starts 0, 1 and then every number after that is the sum of the two before it ($0+1=1$, $1+1=2$, $1+2=3$, and so on). This program prints the first n numbers of the sequence. It keeps the last two numbers in a and b : each time round the loop it prints a , works out the next number ($a + b$) before changing anything, then slides the pair along so a and b become the next two numbers.

Trace this algorithm for $n = 7$, and complete the trace table.



n	a	b	i	next	output
7					
7	0				
7	0	1			
7	0	1	1		
7	0	1	1		0
7	0	1	1	1	
7	1	1	1	1	
7	1	1	1	1	
7	1	1	2	1	
7	1	1	2	1	1
7	1	1	2	2	
7	1	1	2	2	
7	1	2	2	2	
7	1	2	3	2	
7	1	2	3	2	1
7	1	2	3	3	
7	2	2	3	3	
7	2	3	3	3	
7	2	3	4	3	
7	2	3	4	3	2
7	2	3	4	5	
7	3	3	4	5	
7	3	5	4	5	
7	3	5	5	5	
7	3	5	5	5	3
7	3	5	5	8	
7	5	5	5	8	
7	5	8	5	8	
7	5	8	6	8	5
7	5	8	6	13	
7	5	8	6	13	
7	8	8	6	13	
7	8	13	6	13	
7	8	13	7	13	
7	8	13	7	13	8
7	8	13	7	21	
7	13	13	7	21	
7	13	21	7	21	
7	13	21	8	21	

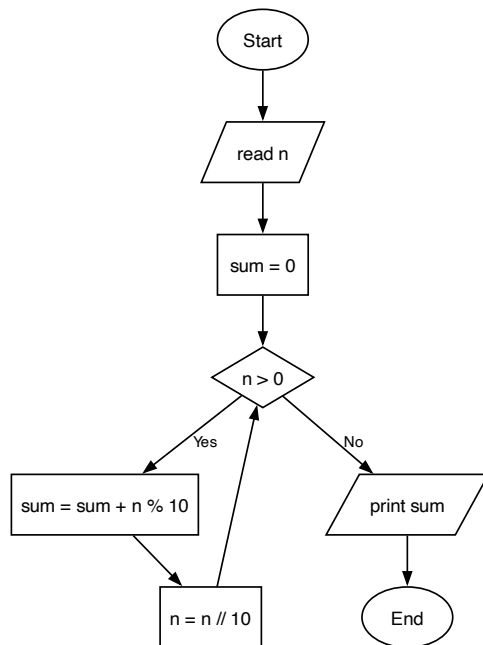
Output: 0, 1, 1, 2, 3, 5, 8

Problem 5.

This program reads a whole number and adds up its digits. It peels off the last digit each time (using the remainder after dividing by 10), adds it to a running total, then drops that digit and repeats until nothing is left.

Trace this algorithm for $n = 4072$, and complete the trace table.

Reminder: $n \% 10$ is the remainder when n is divided by 10 (the last digit of n), and $n // 10$ is n divided by 10 with the remainder thrown away (which drops the last digit) — so $4072 \% 10$ is 2 and $4072 // 10$ is 407.



n	sum	output
4072		
4072	0	
4072	2	
407	2	
407	9	
40	9	
40	9	
4	9	
4	13	
0	13	
0	13	13

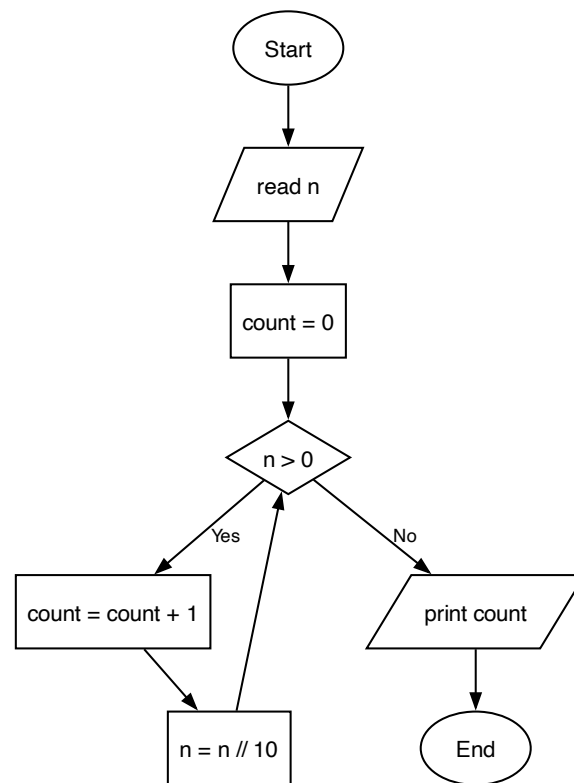
Output: 13

Problem 6.

(i) Expected output for each input:

input n	output
4072	4
538	3
9	1

(ii) Flowchart:

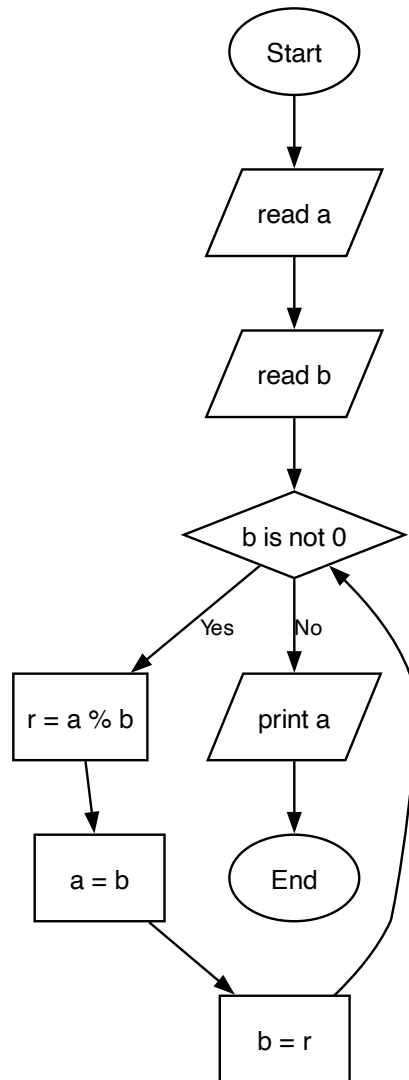


Problem 7.

This is Euclid's method for the greatest common divisor (the largest number that divides both a and b exactly). Each step replaces the pair (a, b) with $(b, \text{remainder of } a \text{ divided by } b)$; when the remainder reaches 0, the answer is the value left in a .

Trace this algorithm for $a = 48, b = 36$, and complete the trace table.

Reminder: $a \% b$ is the remainder when a is divided by b (for example $48 \% 36$ is 12, and $36 \% 12$ is 0).



a	b	r	output
48			
48	36		
48	36	12	
36	36	12	
36	12	12	
36	12	0	
12	12	0	
12	0	0	
12	0	0	12

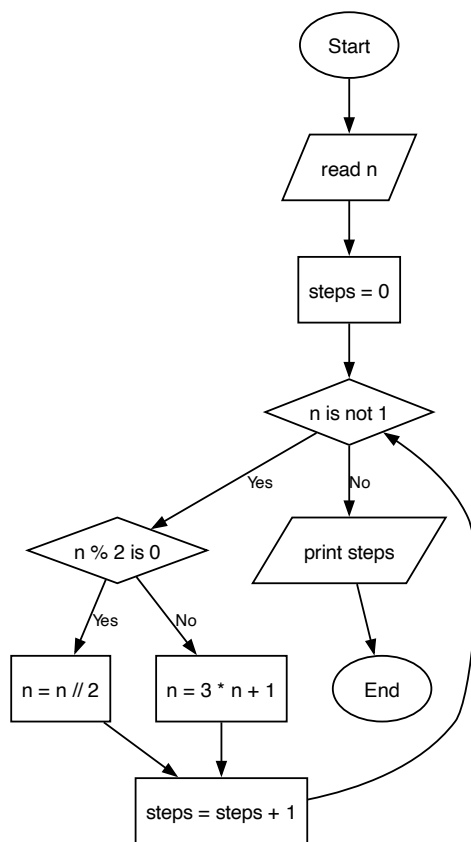
Output: 12

Problem 8.

Start from a whole number n and repeat one rule until you reach 1: if n is even, halve it; if n is odd, replace it with $3 \times n + 1$. This program counts how many steps that takes. (No one has ever found a starting number that fails to reach 1 — but no one has proved it always does, either!)

Trace this algorithm for $n = 6$, and complete the trace table.

Reminder: $n \% 2$ is the remainder when n is divided by 2, so $n \% 2$ is 0 exactly when n is even; $n // 2$ is n halved with any remainder dropped.



n	steps	output
6		
6	0	
3	0	
3	1	
10	1	
10	2	
5	2	
5	3	
16	3	
16	4	
8	4	
8	5	
4	5	
4	6	
2	6	
2	7	
1	7	
1	8	
1	8	8

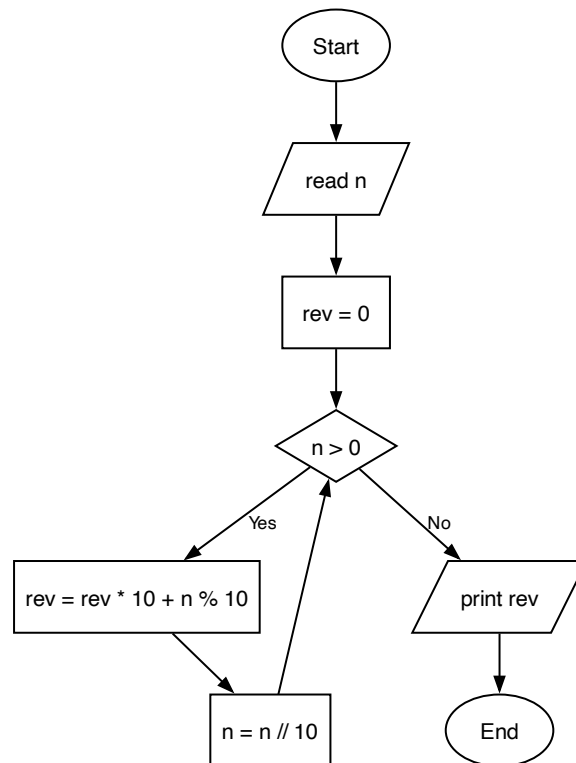
Output: 8

Problem 9.

(i) Expected output for each input:

input n	output
825	528
1900	91
7	7

(ii) Flowchart:

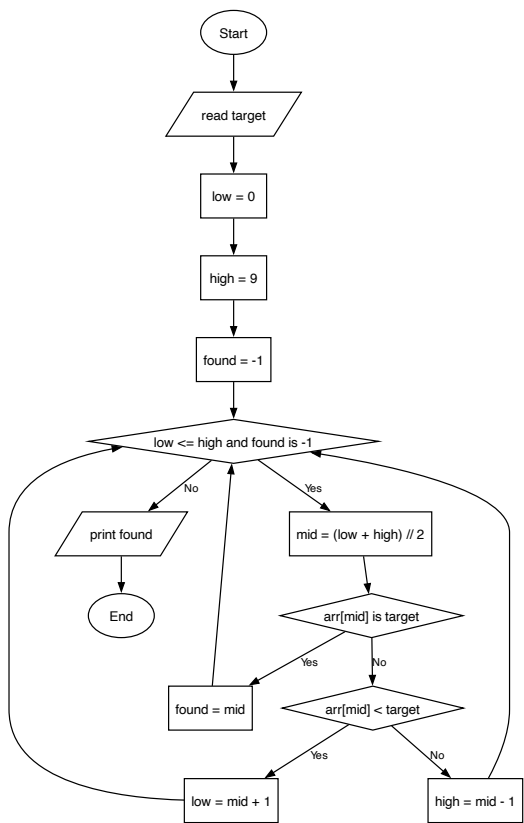


Problem 10.

Binary search finds where a target value sits in a sorted list without checking every item. It tracks a window with two markers, low and high (the positions still worth searching, numbered from 0). Each step looks at the middle position mid: if that item is the target we are done; if it is too small the target must be to the right, so move low up; if it is too big, move high down. found holds the position of the target, or stays -1 until it is located.

Trace this algorithm for target = 23, taking the list arr to be [2, 5, 8, 12, 16, 23, 38, 56, 72, 91], and complete the trace table.

Reminder: arr[mid] means the item at position mid of the list (positions start at 0, so arr[0] is 2); (low + high) // 2 is the middle position, dividing by 2 and dropping any remainder; and the loop keeps going while low <= high and found is still -1.



target	low	high	found	mid	output
23					
23	0				
23	0	9			
23	0	9	-1		
23	0	9	-1	4	
23	5	9	-1	4	
23	5	9	-1	7	
23	5	6	-1	7	
23	5	6	-1	5	
23	5	6	5	5	
23	5	6	5	5	5

Output: 5